

Message Integrity and Message Authentication

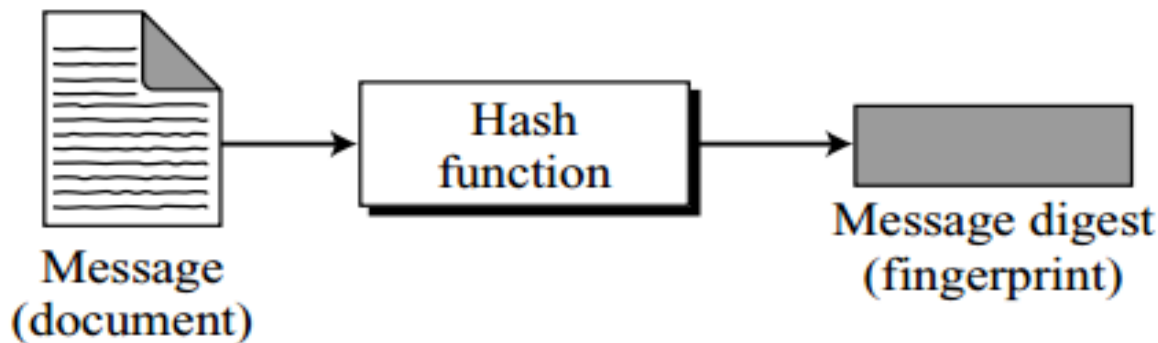
Message Integrity

- Message Integrity confirms that contents of message has not been changed,
- There are occasions where we may not even need secrecy/confidentiality, instead must have integrity.
 - Example: Preparation of important documents, like a will

- One way to preserve the integrity of a document is through the use of a fingerprint.
- The electronic equivalent of the document and fingerprint pair is the message and digest pair.

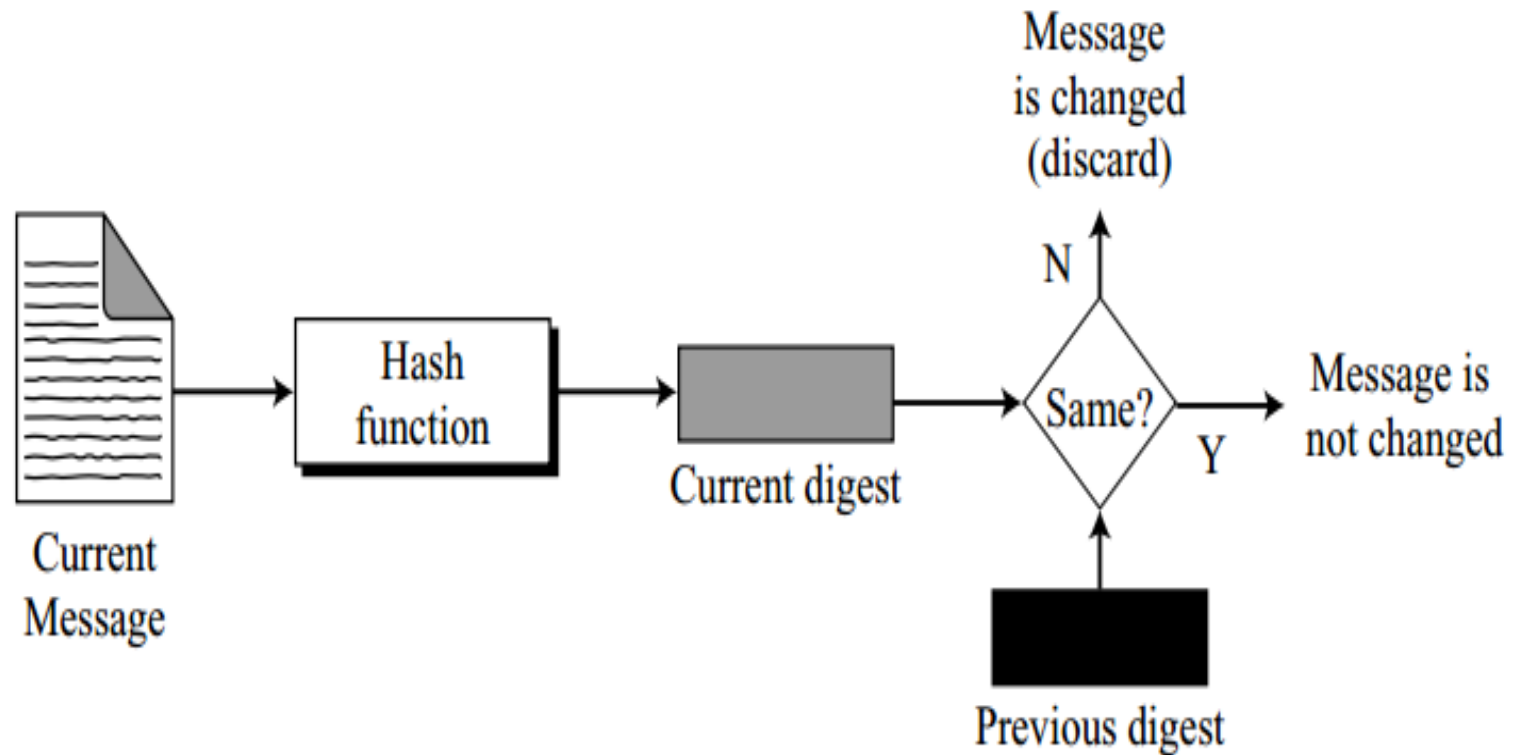
Message and Message Digest

- To preserve the integrity of a message, the message is passed through an algorithm called a cryptographic hash function.

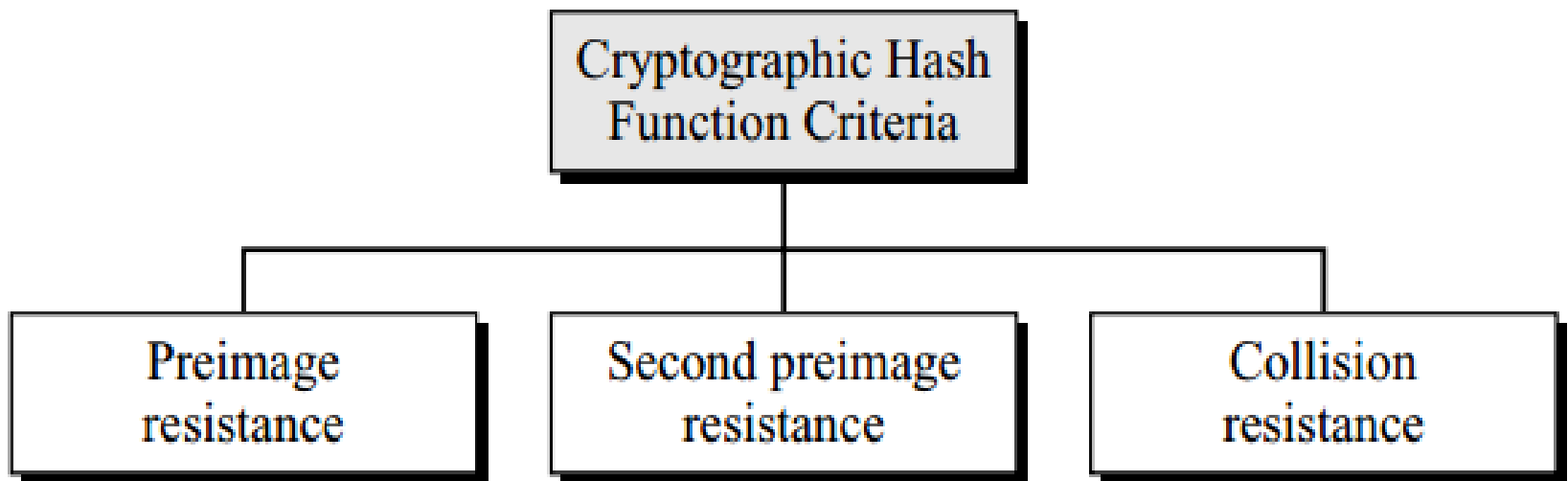


- The two pairs (document/fingerprint) and (message/message digest) are similar
- The document and fingerprint are physically linked together.
- The message and message digest can be unlinked (or sent) separately.
- The message digest needs to be safe from change.

Checking Integrity

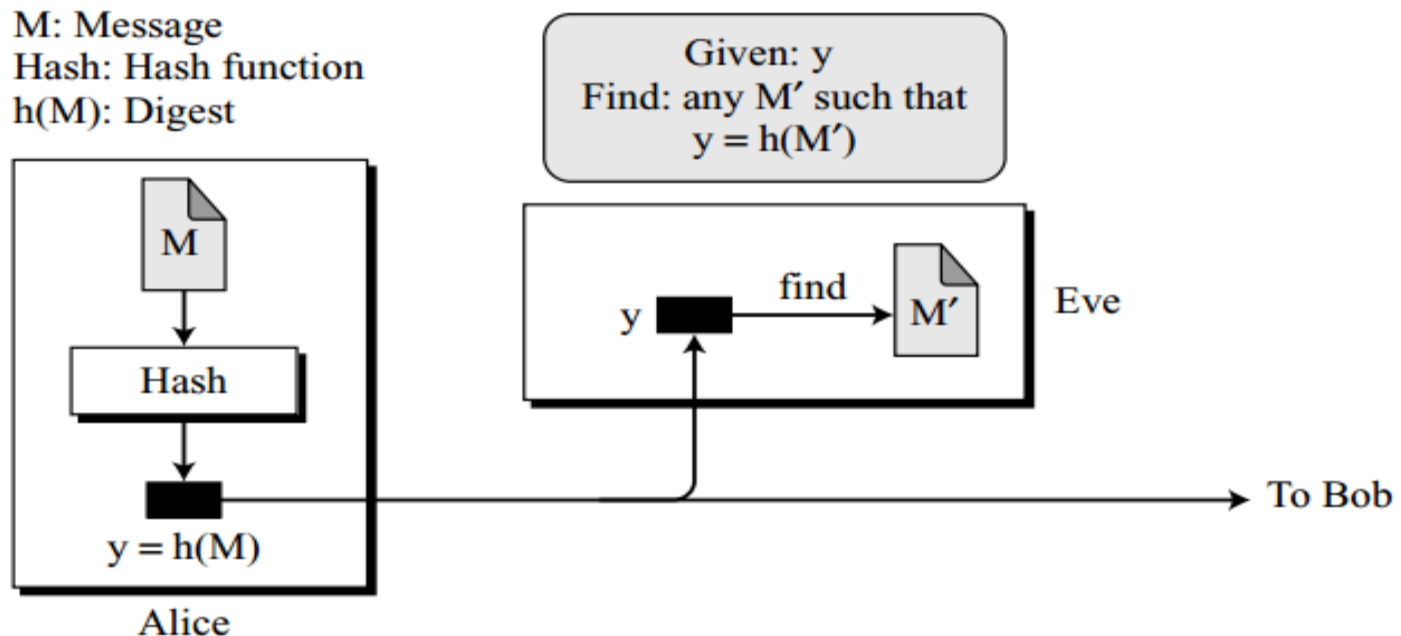


Criteria of cryptographic hash function



Preimage Resistance

- Given a hash function h and $y = h(M)$, it must be extremely difficult for Eve to find any message, M' , such that $y = h(M')$.



Preimage Attack

Given: $y = h(M)$

Find: M' such that $y = h(M')$

Eve can then send M' to Bob pretending it is M .

Second Preimage Resistance

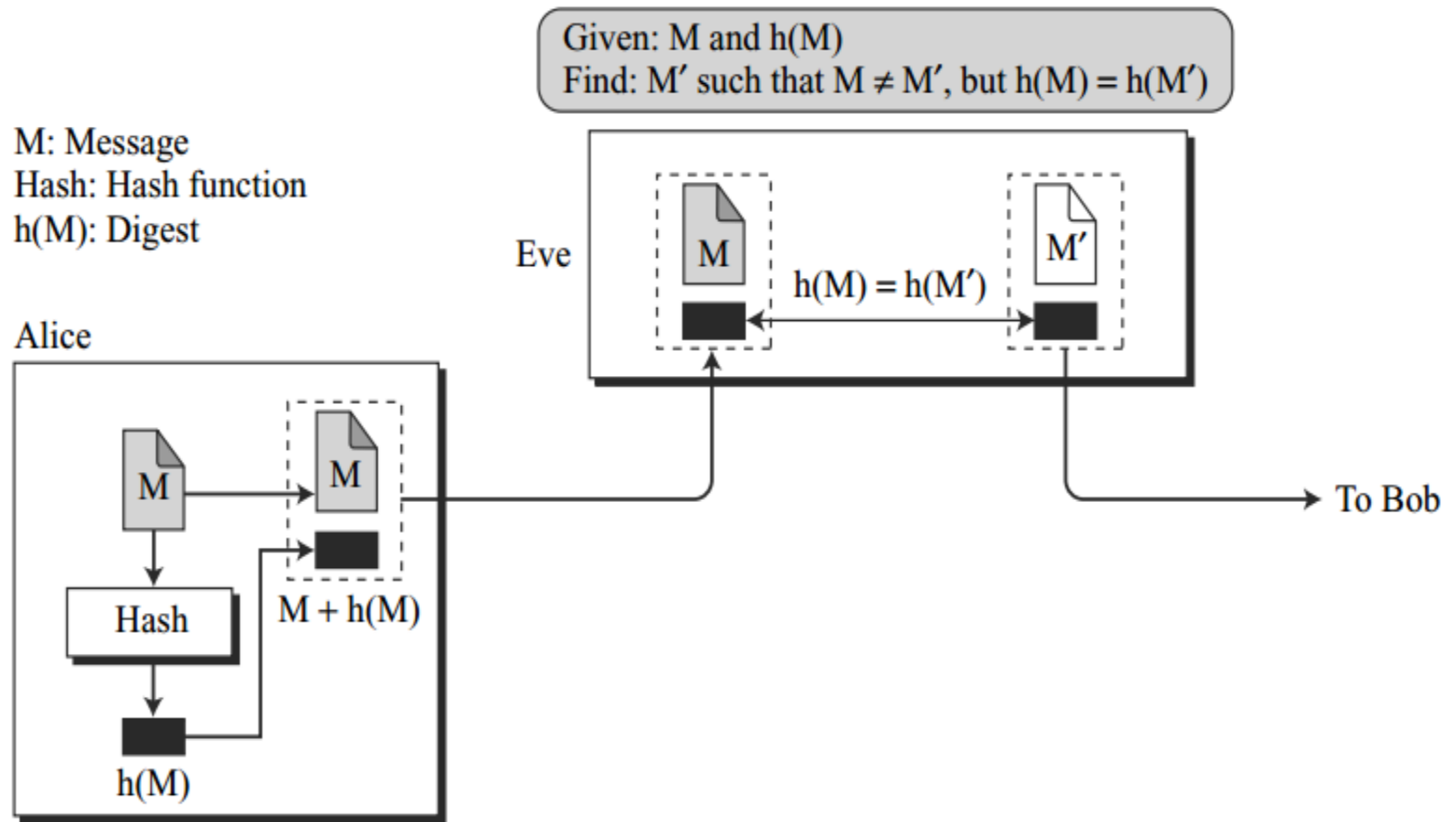
- It ensures that a message cannot easily be forged.
- Given a specific message and its digest, it is impossible (or at least very difficult) to create another message with the same digest.

Second Preimage Attack

Given: M and $h(M)$

Find: $M' \neq M$ such that $h(M) = h(M')$

Second Preimage Resistance

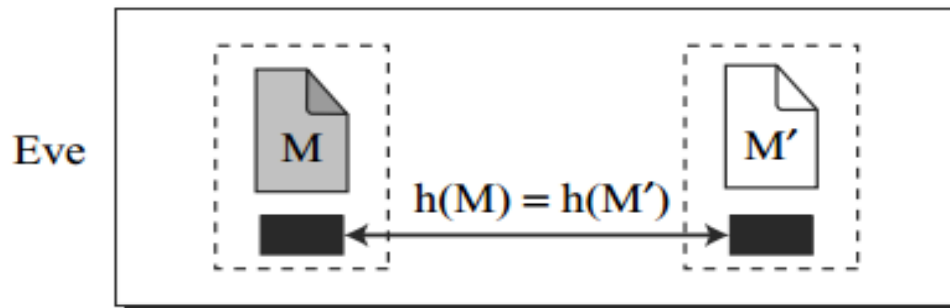


Collision Resistance

- It ensures that an attacker cannot find two messages that hash to the same digest.

M: Message
Hash: Hash function
 $h(M)$: Digest

Find: M and M' such that $M \neq M'$, but $h(M) = h(M')$



Collision Attack

Given: none

Find: $M' \neq M$ such that $h(M) = h(M')$

Random Oracle Model

- It is an mathematical model for a hash function
 - Introduced in 1993 by Bellare and Rogaway
- A function based on this model behaves as per the 3 rules:
 - (1) When a new message of any length is given, the oracle creates and gives a fixed length message digest that is a random string of 0s and 1s. The oracle records the message and the message digest.

Random Oracle Model

- (2) When a message is given for which a digest exists, the oracle simply gives the digest in the record.
- (3) The digest for a new message needs to be chosen independently from all previous digests. This implies that the oracle cannot use a formula or an algorithm to calculate the digest.

Example

- Assume an oracle with a table and a fair coin. The table has two columns.
- The left column shows the messages whose digests have been issued by the oracle.
- The second column lists the digests created for those messages.
- Assume that the digest is always 16 bits regardless of the size of the message.

- Oracle table after issuing the first three digests

<i>Message</i>	<i>Message Digest</i>
4523AB1352CDEF45126	13AB
723BAE38F2AB3457AC	02CA
AB45CD1048765412AAAB6662BE	A38B

Now assume that two events occur:

- a. The message AB1234CD8765BDAD is given for digest calculation.

The oracle checks its table. This message is not in the table, so the oracle flips its coin 16 times. Assume that result is HHTHHHTTHTHHTTTH, in which the letter H represents heads and the letter T represents tails. The oracle interprets H as a 1-bit and T as a 0-bit and gives 1101110010110001 in binary, or DCB1 in hexadecimal

- Oracle table after issuing the fourth digest

<i>Message</i>	<i>Message Digest</i>
4523AB1352CDEF45126	13AB
723BAE38F2AB3457AC	02CA
AB1234CD8765BDAD	DCB1
AB45CD1048765412AAAB6662BE	A38B

b. The message 4523AB1352CDEF45126 is given for digest calculation.

- The oracle checks its table and finds that there is a digest for this message in the table (first row).
- The oracle simply gives the corresponding digest (13AB).

Pigeonhole Principle

- if n pigeonholes are occupied by $n + 1$ pigeons, then at least one pigeonhole is occupied by two pigeons.
- The generalized version of the pigeonhole principle is that if n pigeonholes are occupied by $kn + 1$ pigeons, then at least one pigeonhole is occupied by $k + 1$ pigeons.

Pigeonhole Principle

- Hashing indicates that, the digest should be shorter than the message
 - according to the pigeonhole principle there can be collisions.
 - there are some digests that correspond to more than one message

Example

- Assume that the messages in a hash function are 6 bits long and the digests are only 4 bits long.
- Then the possible number of digests (pigeonholes) is $2^4 = 16$
- The possible number of messages (pigeons) is $2^6 = 64$.
- This means $n = 16$ and $kn + 1 = 64$, so k is larger than 3.

conclusion : at least one digest corresponds to four ($k + 1$) messages.

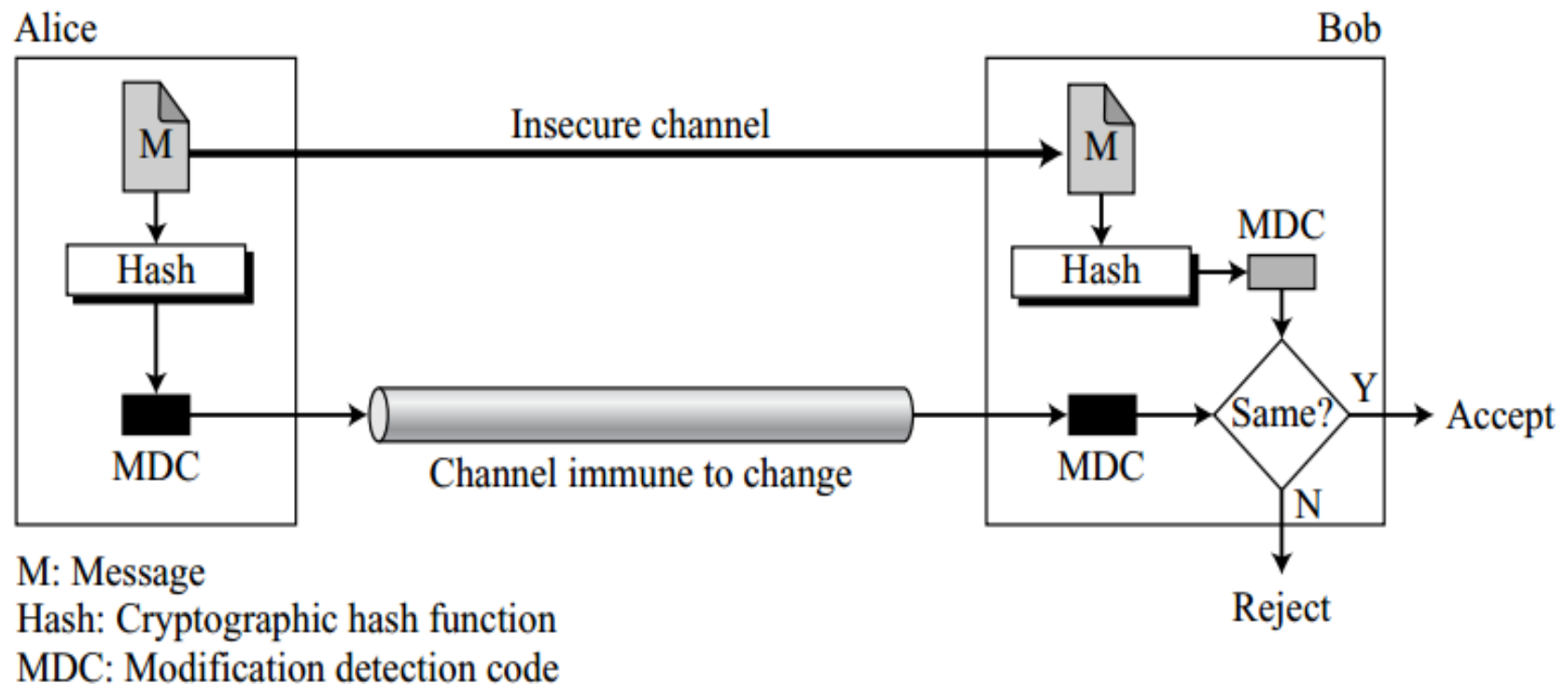
Message Authentication

Message Authentication

- A message digest does not authenticate the sender of the message.
- The digest created by a cryptographic hash function is normally called a **modification detection code (MDC)**.
- Message authentication (data origin authentication) is achieved through **message authentication code (MAC)**.

Modification Detection Code

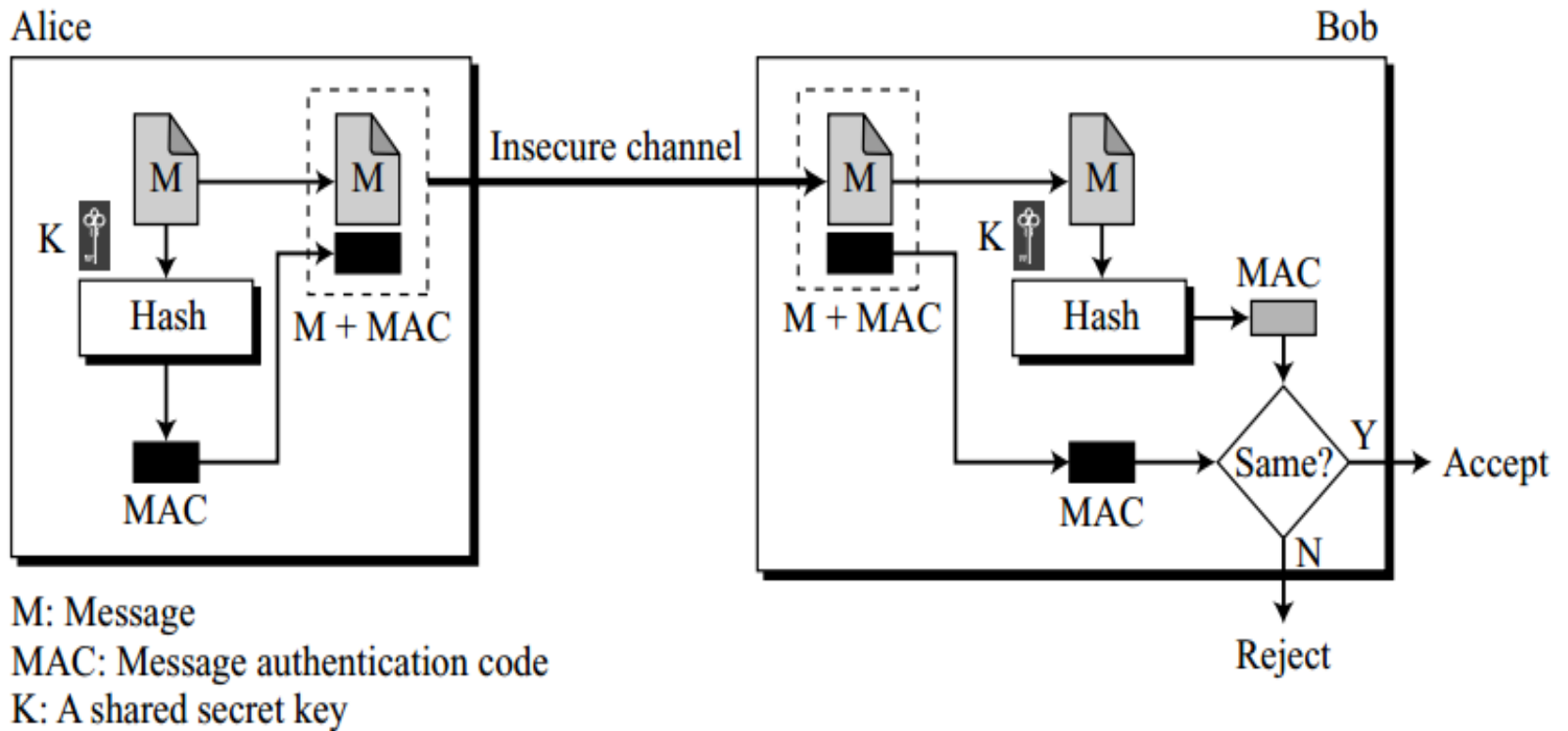
- It is a message digest that can prove the integrity of the message



Message Authentication Code (MAC)

- To ensure the integrity of the message and the data origin authentication, we need MAC.
- The difference between a MDC and a MAC is that the MAC includes a **secret** (key) between Alice and Bob
 - Both message and the MAC can be sent on the same insecure channel, unlike the MDC

Message Authentication Code (MAC)

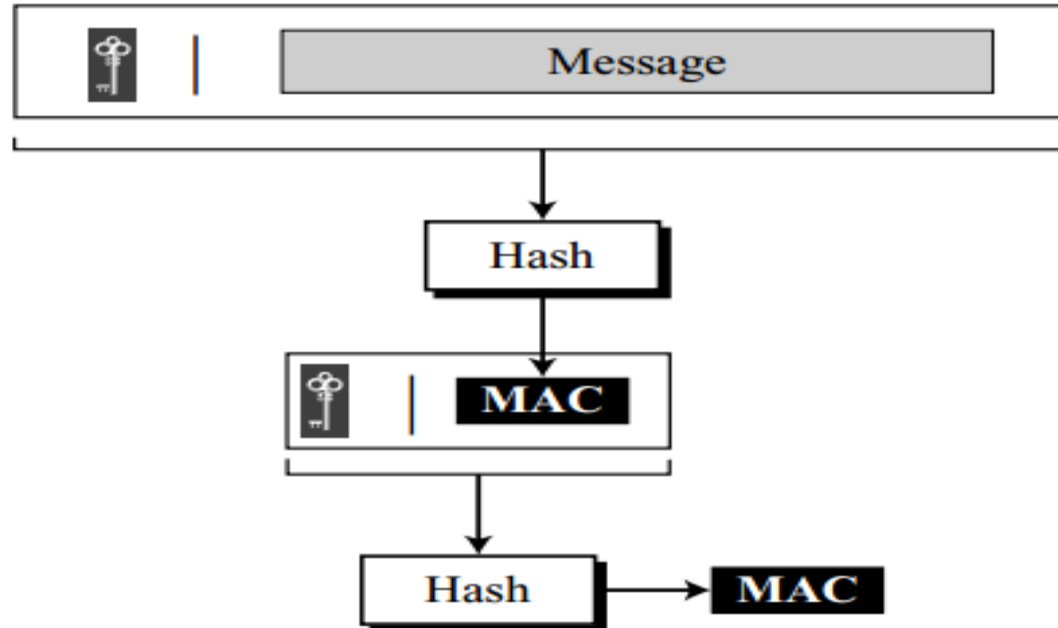


Security of a MAC

- The security of a MAC depends on the security of the underlying hash algorithm.
- The size of the key is normally very large in a MAC
 - Using preimage attack, the attacker can find the key and successfully replace the message with a forged one.
 - Given some pairs of messages and their MACs, the attacker can manipulate them to come up with a new message and its MAC.

Nested MAC

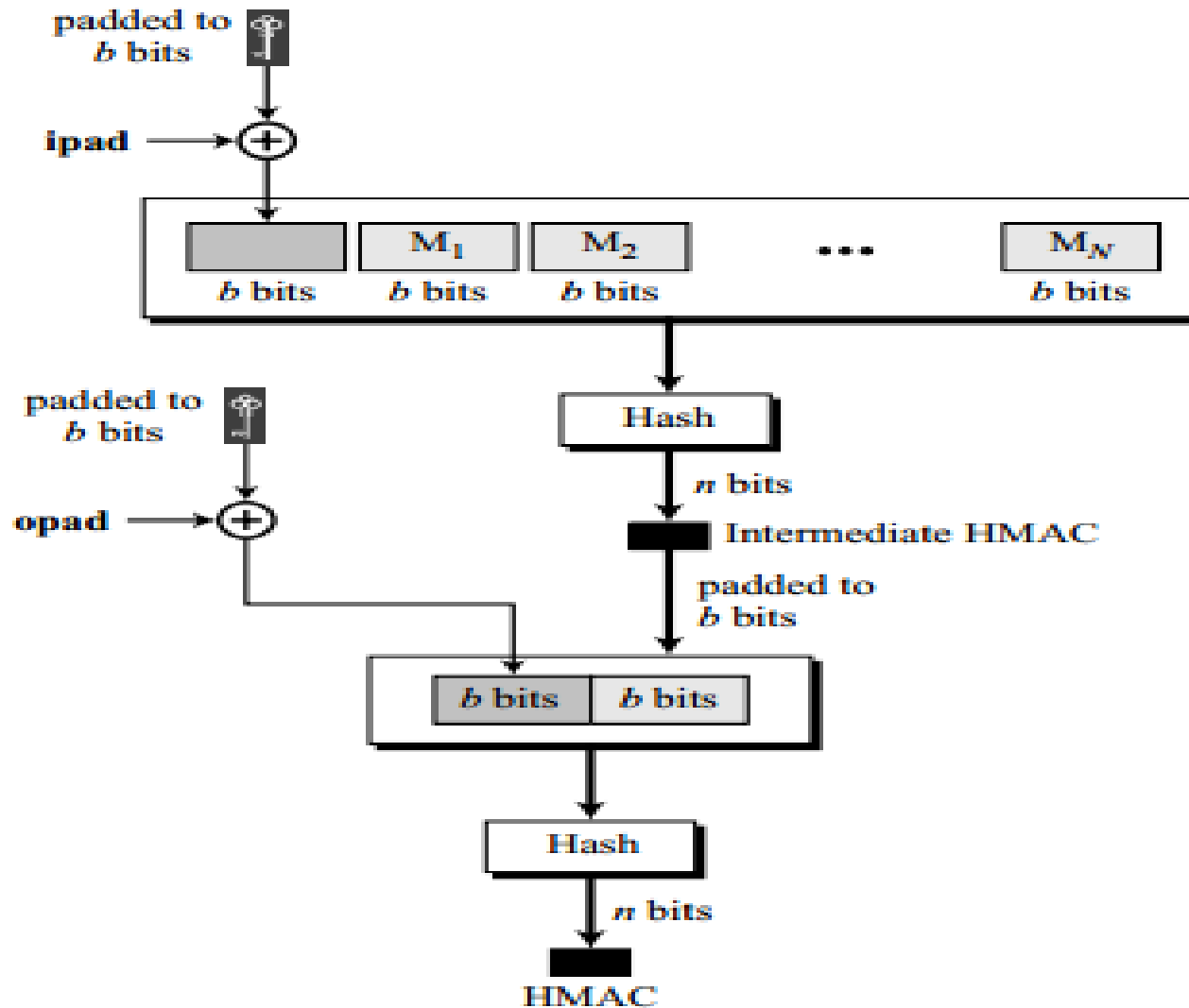
- To improve the security of a MAC, nested MACs were designed in which hashing is done in two steps.



HMAC

- NIST has issued a standard (FIPS 198) for a nested MAC that is often referred to as HMAC (**hashed MAC**)
- HMAC is much more complex than the nested MAC, with additional features, such as **padding**

HMAC Operation



HMAC Operation

1. The message is divided into N blocks, each of b bits.
2. The secret key is left-padded with 0's to create a b -bit key. Note that it is recommended that the secret key (before padding) be longer than n bits, where n is the size of the HMAC.
3. The result of step 2 is exclusive-ored with a constant called **ipad (input pad)** to create a b -bit block. The value of **ipad** is the $b/8$ repetition of the sequence 00110110 (36 in hexadecimal).
4. The resulting block is prepended to the N -block message. The result is $N + 1$ blocks.
5. The result of step 4 is hashed to create an n -bit digest. We call the digest the intermediate HMAC.

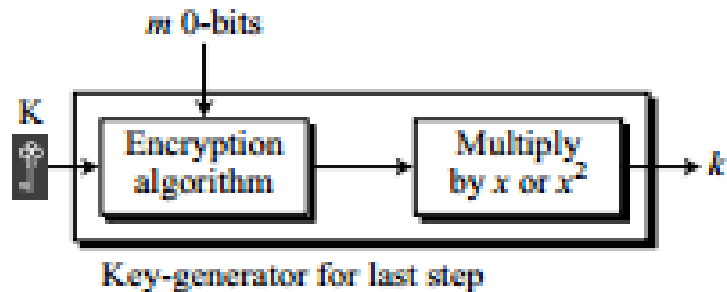
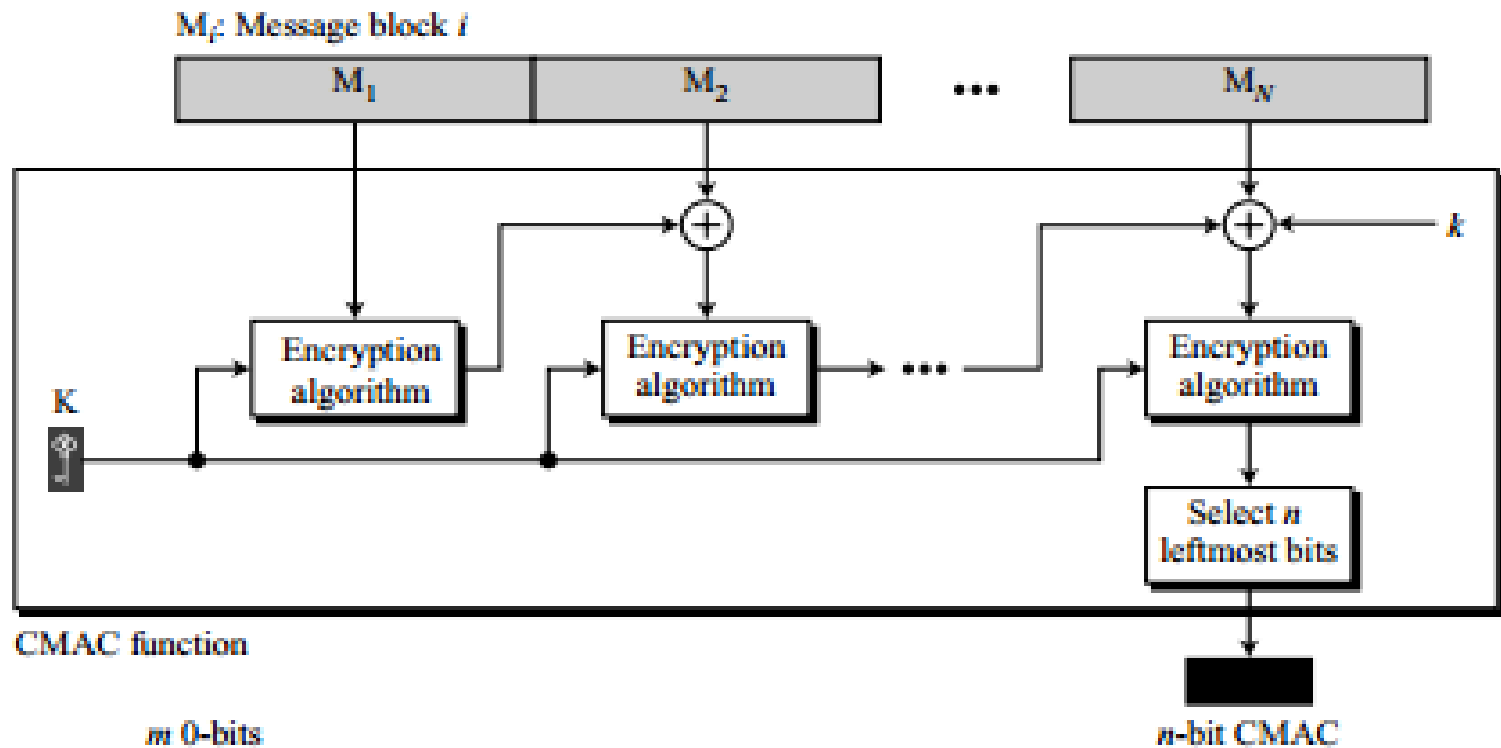
HMAC Operation

6. The intermediate n -bit HMAC is left padded with 0s to make a b -bit block.
7. Steps 2 and 3 are repeated by a different constant opad (**output pad**). The value of opad is the $b/8$ repetition of the sequence 01011100 (5C in hexadecimal).
8. The result of step 7 is prepended to the block of step 6.
9. The result of step 8 is hashed with the same hashing algorithm to create the final n -bit HMAC.

CMAC

- NIST has also defined a standard (FIPS 113) called Data Authentication Algorithm, known as, **CMAC or CBCMAC**.
- Similar to the cipher block chaining (CBC) mode for symmetric-key encipherment.
- The idea is to create one block of MAC from N blocks of plaintext using a symmetric-key cipher N times.

CMAC



CMAC

- In addition to the symmetric key, K , CMAC also uses another key, k (**applied only at the last step**)
- This key is derived from the encryption algorithm with plaintext of m 0-bits using the cipher key, K .
- The result is then multiplied by x if no padding is applied and multiplied by x^2 if padding is applied.
- The multiplication is in $GF(2^m)$ with the irreducible polynomial of degree m